

PHY607

Gabe Suarez

September 2026

1 Abstract

This report/document is for the first project in Physics 607, Fall 2026. In this project, we were each tasked with choosing one ordinary differential equation (preferably of low order) and one function (again, must be Riemann integrable) to integrate.

The differential equation of choice was the Kepler differential equation $\ddot{r} = \frac{L^2}{\mu^2 r^3} - \frac{GM\mu}{r^2}$. We will discuss the physical origin, and details of numerical solvers used, including limitations.

For the integral, I chose the main integral which appears in the Veneziano amplitude, $\int_0^1 dx x^{(s-1)}(1-x)^{(t-1)}$, which in mathematics is the Euler Beta function, $B(s, t) = \frac{\Gamma(s)\Gamma(t)}{\Gamma(s+t)}$. Specifically, since this function is convergent and finite for all real s, t if we restrict $s > 0, t > 0$, we only integrate over real $s > 0, t > 0$. This is a crucial detail since in the Veneziano amplitude, the excited string states are represented by the poles in the amplitude, and occur at real $s < 0, t < 0$. If one wanted to integrate x , over such s, t , an analytic continuation of $B(s, t)$ must be performed. This can be done by analytically continuing $\Gamma(x)$, using its recurrence relation, or other representations. However, for our purposes, it suffices to say that $B(s, t)$ becomes a meromorphic function, which are never Riemann integrable at the poles. We could, then, integrate some fancy contours in the complex s, t plane, but that is outside the scope of this project.

2 Introduction

2.1 Kepler ODE

The Kepler differential equation represents a classical problem in classical Newtonian mechanics: how to describe the motion of two bodies which orbit each other? To cut it short, let's answer this by prescribing a Lagrangian to the system. Let $r_1, \theta_1, r_2, \theta_2$ be the initial positions of the two bodies, with their respective initial conditions (such as $\dot{r}_{1_0}, \dot{r}_{2_0}$). Go to relative coordinates defined by $r = r_1 - r_2$, let $\mu = \frac{m_1 m_2}{m_1 + m_2}$. Write the Lagrangian of the system in these

coordinates, giving:

$$L = \frac{1}{2}\mu(\dot{r}^2 + r^2\dot{\theta}^2) + \frac{Gm_1m_2}{r} \quad (1)$$

Applying the Euler-Lagrange differential operator w.r.t to r, θ , to L :

$$\frac{d}{dt} \frac{\partial L}{\partial \dot{r}} - \frac{\partial L}{\partial r} = 0 = \mu\ddot{r} - \mu r\dot{\theta}^2 - \frac{Gm_1m_2}{r^2} \quad (2)$$

$$\frac{d}{dt} \frac{\partial L}{\partial \dot{\theta}} - \frac{\partial L}{\partial \theta} = 0 = \frac{d}{dt}(\mu r^2\dot{\theta}) = \frac{d}{dt}l \quad (3)$$

Which, if we then use l to re-arrange the first equation, use $m_1 + m_2 = M, \mu$ above, we get:

$$\mu\ddot{r} = \frac{l^2}{\mu r^3} - \frac{GM\mu}{r^2} \quad (4)$$

Which is the equation in question, to solve numerically.

Note the conservation of l . We could, in the implementation of the differential equation, track $\mu r^2\dot{\theta}$ and ensure it's constant, as a consistency check. Furthermore, we can take various asymptotics to test this equation, to test the behavior. We will discuss this later on in analysis.

2.2 Veneziano Amplitude & Beta

Deriving the Veneziano amplitude is a bit more involved-and perhaps, if I may say, not super insightful. But, we do it nevertheless, for completeness. Just be prepared for some string theory.

First, the idea. Take a string, parametrized by two coordinates σ, t . Give your favorite boundary conditions to σ such as $\sigma \in [0, 2\pi]$, for the closed string, or $\sigma \in [0, \pi]$, or whichever normalized length you wish, for the open string. Of course, $t \in R_+$.

The idea is this string is embedded in some target space M , which we encode by the embedding field $X(\sigma, t)$. Enforce those same boundary conditions on X , as well as the typical Neumann/Dirichlet, with the appropriate vanishing of δX , on the action. Again, not writing these details-since that'd take pages, we focus purely on the worldsheet.

The worldsheet is, up to a conformal transformation, (which we can always make) a 2 dimensional Riemann surface swept out by the propagating string through time. Let this Riemann surface be represented by Σ .

The Veneziano amplitude concerns the tree-level amplitude of four open-strings (it does change, if you chose closed strings - to the Veneziano-Shapiro amplitude). We will represent this geometrically by first realizing that, this amplitude is generated by a path integral. You may say-path integral over what? Well, precisely, the generic path integral, after gauge fixing with diffeomorphism+Weyl invariance, becomes a path integral over 2d topologies, and the embedding field:

$$Z \sim \sum_{\text{Topologies } T} \int dX e^{-S_{Polyakov}} \quad (5)$$

In doing this, though again I'm not showing it in every little detail, we'd have to integrate over the Ricci scalar of the worldsheet, which becomes a coupling constant to the power of the Euler characteristic of T , in the front of the path integral, and... some more details that truly are not important for this particular part.

If we want tree level scattering, we again use another deep fact that in 2d: every surface S , is characterized by a choice topology T , and it's boundaries. Equivalently, choosing cross cap count n_c , genus g , boundary count b , with Euler characteristic χ constraining their relation. In doing this, we take $n_c = 0, g = 0, b = 1$ - namely the disc, since we want the most basic topology, and a boundary to insert operators on.

This reduces the path integral, if we insert 4 operators, which represent open strings to: $Z_{Veneziano} = \int dX e^{-S} V_1 \dots V_4$. Generically, since we want to consider asymptotic states, we take each open string to be in its ground state, which then means the vertex operators are represented by plane waves - $V_i =: e^{ik_i \cdot X_i(z_i)}$, where $::$ denotes normal ordering, with momentum k of the string state i , and embedding field $X(z)$:

$$Z_{Veneziano} = \int dX e^{-S} : e^{ikX_1(z_1)} :: e^{ikX_2(z_2)} :: e^{ikX_3(z_3)} :: e^{ikX_4(z_4)} : \quad (6)$$

Now for another fun fact. Polyakov's action is a totally free field theory, which means that these asymptotic string states won't interact, meaning we can contract this above using Wick's theorem into a product of Gaussian integrals over the typical propagator for X . The propagator is best derived using the OPE for X , which is approximately $\partial X(z) \partial X(w) \sim \frac{1}{(z-w)^2}$, integrate that twice to get $\sim \ln((z-w)^2)$. And then of course since $e^{\ln(x)} = x$, then we just get a sum over powers of $-(z-w)^y$, where y will be dot products of $k_i k_j$ etc. . Going through it carefully, using the exact $\langle XX' \rangle$ OPE, and realizing that the X zero mode provides momentum conversation, we get:

$$Z_{Veneziano} = \delta^D \left(\sum_j k_j^\mu \right) \prod_{i>j} |z_i - z_j|^{(k_i \cdot k_j)} \quad \mu = 0, \dots, D-1 \quad (7)$$

Where δ^D is represented as $\delta^D(\cdot) = \prod_{i=0}^{D-1} \delta^i(\cdot)$.

Now, we use another deep fact. Since the disc is automatically conformally invariant under $SL(2, \mathbb{R})$, and since $SL(2, \mathbb{R})$ is generated by the three Virasoro generators (L_{-1}, L_0, L_1) , we can then fix three of the four z_i by acting with independent $g \in SL(2, \mathbb{R})$ on each z_i , $g : z_i \rightarrow z'_i$. By convention, we take $z_0 = 0, z_1 = z, z_2 = 1, z_3 = \infty$. In doing this, we're left with z_i free between z_0 and z_2 , so we must integrate over it:

$$Z_{Veneziano} = \delta^D \left(\sum_j k_j^\mu \right) \int_0^1 dz z^{k_0 \cdot k_1} (1-z)^{k_1 \cdot k_2} \quad (8)$$

Now at this point, we define Mandelstam variables in order to write the above in a familiar form, to integrate it! The genuinely non-obvious part here is

the mass-momentum relation for the specific bosonic open string in the ground state. It satisfies $m^2 = -1$. To be clear, this would require one to write out Polyakov's action, solve the equation of motion (just a free wave thankfully), introduce canonical quantization between the string embedding modes α , define the ground state as that annihilated as $L_0|0\rangle = 0$ and then solve for the L_0 eigenvalue in it's mode expansion representation of the mode operators. Writing that all out is not necessary. Let's continue. Using the non-obvious bosonic open-string ground state mass relation $m^2 = -1$, then $m^2 = -k^\mu k_\mu = -1$ or $k^\mu k_\mu = 1$, for all open string insertion states. That implies, where $s = (k_0 + k_1)^2$, $t = (k_1 + k_2)^2$, when we compute $k_0 \cdot k_1 = k_0^2 + k_1^2 + 2k_0 \cdot k_1 = 2 + 2k_0 \cdot k_1 = -s/2 - 1$. Similiarly $k_1 \cdot k_2 = k_1^2 + k_2^2 + 2k_1 \cdot k_2 = 2 + 2k_1 \cdot k_2 = -t/2 - 1$, meaning we can rewrite the above as:

$$Z_{Veneziano} = \delta^D(\sum_j k_j^\mu) \int_0^1 dz z^{-1-s/2} (1-z)^{-1-t/2} \quad (9)$$

Then just define $s' = -1 - s/2 + 1 = -s/2$, $t' = -1 - t/2 + 1 = -t/2$

$$Z_{Veneziano} = \delta^D(\sum_j k_j^\mu) \int_0^1 dz z^{s'-1} (1-z)^{t'-1} \quad (10)$$

And since we're doing a path integral we have to consider all configurations—meaning that our choice couldn't have made a difference, and thus we must sum over all nontrivial permutations of the states. There will be 2 more, just like in QFT. We have the s,t channel amplitude, but need t,u, and s,u. But, of course, we only care about the above result since we need to integrate THAT thing, and not the full amplitude. Thus, we can conclude:

$$Z_{Veneziano} \sim B(s, t) = \int_0^1 dz z^{s'-1} (1-z)^{t'-1} , \quad s' > 0, t' > 0 \quad (11)$$

3 Numerical Solvers

3.1 ODE Solvers

The attached python script *phy604proj1_ode.py* solves the first ODE. It first prompts the user to input the following:

- First input ODE solver type - either Euler semi-implicit, or Runge-Kutta 4th order (RK4)
- Initial conditions (M, μ, r_0, v_0 , etc.) ; θ_0 is chosen as 0
- Time step count (N), step size in time (Δt)

Writing the update equations, in both versions, out explicitly:

$$\text{Euler Semi-Implicit:} \quad (12)$$

$$\dot{r}_{n+1} = r_n + \ddot{r}_n \Delta t = r_n + \left(\frac{l^2}{\mu^2 r_n^3} - \frac{GM}{r_n^2} \right) \Delta t \quad (13)$$

$$r_{n+1} = r_n + \dot{r}_{n+1} \Delta t \quad (14)$$

$$\theta_{n+1} = \theta_n + \dot{\theta}_n \Delta t = \theta_0 + \frac{l}{\mu r_n^2} \Delta t \quad (15)$$

The RK4 equations are a bit hairier, since RK4 is designed for first order ODEs, but we can play a game where we split $\ddot{r} = \dot{v}$ by $v = \dot{r}$. We then get two update equations for the RK4 scheme:

$$\text{Runge-Kutta 4th Order:} \quad (16)$$

$$\text{Iteratively take } r \rightarrow r + a_i/2, v \rightarrow v + b_i/2, \theta \rightarrow \theta + c_i/2, t \rightarrow t + \Delta t/2 \quad (17)$$

$$a_1 = \Delta t v_n \quad b_1 = \Delta t \dot{v} = \Delta t \ddot{r}|_{r=r_n} \quad c_1 = \Delta t (l/(\mu r_n^2)) \quad (18)$$

$$a_2 = \Delta t (v_n + b_1/2) \quad b_2 = \Delta t (\ddot{r}|_{r=(r_n+a_1/2)}) \quad c_2 = \Delta t (l/(\mu (r_n + a_1/2)^2)) \quad (19)$$

$$a_3 = \Delta t (v_n + b_2/2) \quad b_3 = \Delta t (\ddot{r}|_{r=(r_n+a_2/2)}) \quad c_3 = \Delta t (l/(\mu (r_n + a_2/2)^2)) \quad (20)$$

$$a_4 = \Delta t (v_n + b_3) \quad b_4 = \Delta t (\ddot{r}|_{r=(r_n+a_3)}) \quad c_4 = \Delta t (l/(\mu (r_n + a_3)^2)) \quad (21)$$

$$r_n = r_i + \frac{1}{6}(a_1 + 2a_2 + 2a_3 + a_4) \quad (22)$$

$$v_{n+1} = v_n + \frac{1}{6}(b_1 + 2b_2 + 2b_3 + b_4) \quad (23)$$

$$\theta_{n+1} = \theta_n + \frac{1}{6}(c_1 + 2c_2 + 2c_3 + c_4) \quad (24)$$

3.2 Analysis of ODE Correctness

The other note is that the above is exact math-but can I implement these exactly into python without error? Well, yes as the author I do believe so. But you be the judge, please. Here is the evidence. It's two fold. Firstly, the ODE we chose is exactly integrable (in the Liouville sense). That being said, I really don't want to write it all out, but for the fun of it, we might as well. Actually, no, I take it back. No more time wasting. The full derivation for $r(\theta)$ are all derived in Goldstein's Classical Mechanics chapter 3. We use Goldstein equations 3.15,

3.41, 3.54, 3.55:

$$E = \frac{1}{2}\mu\dot{r}^2 + \frac{1}{2}\mu r^2\dot{\theta}^2 - \frac{GM\mu}{r} \quad (25)$$

$$\theta - \theta' = -\arccos\left(\frac{\frac{l^2}{GM\mu^2 r} - 1}{\sqrt{1 + \frac{2El^2}{G^2 M^2 \mu^3}}}\right) \quad (26)$$

$$r(\theta) = \frac{\frac{l^2}{GM\mu^2}}{1 + \sqrt{1 + \frac{2El^2}{G^2 M^2 \mu^3}} \cos(\theta - \theta_0)} \quad (27)$$

There are multiple tests we can do immediately with these. Firstly, since E is a constant, we can compute it for all t , and see it's numerical drift/if our equations are, at minimum, correct. We will use E as our behavior parameter for this ODE. Furthermore, we can then check our $r(t)$, at least up to the accuracy of θ , since we evolved θ in the RK4 sense. Still, that's only one depth of numerical approximation, rather than two, which is what $v, \dot{v}$ go under.

For pure numerical comparison, we need to see if our results are consistent with trustable solvers-such as SciPy's integral and ODE solver. Furthermore, we thankfully can do this easily in python. We literally input the integral into SciPy for a grid over s, t and take the difference between our computed integral values at s, t and SciPy's, then normalize the difference, to get the % difference. Below are the results.

3.3 Integration Solvers

The attached python script *phy604proj1_int.py* solves the Beta integral, over $s > 0, t > 0$. It first prompts the user to input the following:

- Integration method - Riemann, Trapezoid, Simpson
- Initial conditions - value of s,t
- Count of integration points between [0,1]
- s,t grid sampling point quantity
- s,t grid range

From there, everything is handled.
Just as with the ODE setup, the integration setup:

$$\text{Riemannian: } \int_a^b f(x)dx \approx \sum_{k=0}^{N-1} dx f(a + k \, dx) ; \, dx = (b - a)/N \quad (28)$$

$$\text{Trapezoid: } \int_a^b f(x)dx \approx \sum_{k=0}^{N-1} (f(a + (k + 1) \, dx) + f(a + k \, dx)) \frac{dx}{2} \quad (29)$$

$$\text{Simpson: } \int_a^b f(x)dx \approx \quad (30)$$

$$(f(a) + \sum_{k_{\text{odd}}=1}^{N-1} 4f(a + kdx) + \sum_{k_{\text{even}}=2}^{N-2} 2f(a + kdx) + f(b)) \frac{dx}{3}; \, f(b) = f(a + Ndx) \quad (31)$$

3.4 Analysis of Integration Correctness

Thankfully, compared to the ODE, the Euler Beta function is a walk in the park. Well, to be precise, it is already a generally well-behaved meromorphic function for all $s, t \in \mathbb{C}$, with integer spaced poles at $\Re(s + t) \in \mathbb{Z}_{<0}$. But we don't even have to worry about that part. So we're dealing with a totally analytic function $\forall z, s, t \in \mathbb{R}_{>0}$, which has a convergent finite value over it's whole domain. The only real "correctness" we need to consider then, is the inherent error in the numerical integration method, for some given initial conditions, versus SciPy's $B(s, t)$. We do this in the "Example Solutions - Integration" section.

4 Example Solutions - ODE

4.1 Figures

All plots in this section are for RK4-all Euler based solver plots are in the appendix E.1 - Euler ODE. Let's solve for a few examples. We can do this by mapping to the examples that Goldstein gives, and by thinking on our own. A basic start is we want $r = r_0$ for all time t . This means

$$\dot{r} = \ddot{r} = 0 \implies v_0 = 0, \, r_0 = \frac{l^2}{GM\mu^2} \quad (32)$$

If we want an eccentricity e , per Goldstein, we require the following:

$$r_0 = \frac{l^2}{GM\mu^2} \frac{1}{1 + e} \quad (33)$$

$$e = \left| \frac{r_0^3 \dot{\theta}_0^2}{GM} - 1 \right| \quad (34)$$

Where the second equation comes from solving for E given $v_0 = 0$, inverting equations 3.60-3.62 for e . Using this we can solve for various orbits.

Circular orbits. Figure 1. Take $r_0 = 1, v_0 = 0, G = 1, M = 1, m = 0.1, \dot{\theta} = 1$ in the script, and run it (or any various combination of these which satisfy $\dot{r} = \ddot{r} = 0$). Take some large amount of steps N , reasonable timestep $dt \in [0.1, 0.0001]$, where your $t_{\max} = Ndt$. The code will generate 5 plots, labeled appropriately. Units are on axes as usual. The first plot gives $\theta(t)$ versus $r(t)$. The second $y(t)$ versus $x(t)$, to parametrically plot numerical $r(t), \theta(t)$. The third gives analytic (again up to theta RK4 step) $y(t)$ versus $x(t)$.¹ The fourth is the difference between the third and second. The fifth plot is energy E versus t .²

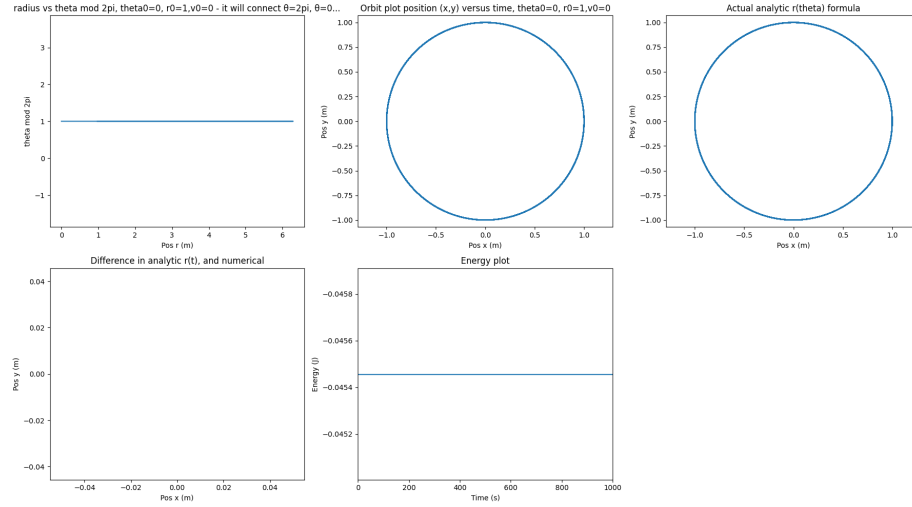


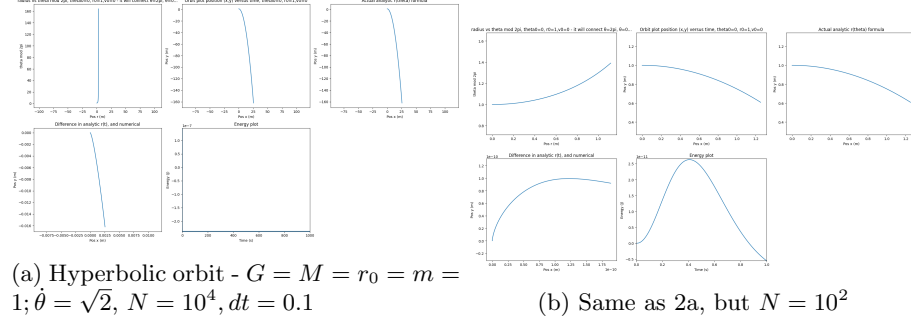
Figure 1: Circular orbit - $G = M = r_0 = \dot{\theta}_0 = 1; m = 0.1, N = 10^4, dt = 0.1$

Hyperbola. For a hyperbolic orbit, we require $E > 0, e > 1$. Here, we can satisfy such equations by setting $E = \epsilon, \frac{r_0^3 \dot{\theta}_0^2}{GM} = 2 + \epsilon, \epsilon > 0$. From eq.29 above for E we see that we can uniquely solve for $\mu, \dot{r}, \dot{\theta}_0, G, M$. Perhaps more realistically, we can demand $G = M = m = 1$ at some r_0 , then solve for $\dot{\theta}_0$, setting $v_0 = 0$. In doing this, we get (literally by then setting eq. 29 to 0), $\dot{\theta} > \sqrt{\frac{2}{r_0^3}}$. This of course satisfies the e requirement. Taking then, $G = M = m = \mu = 1, v_0 = 0$, taking $r_0 = 1$ then we arbitrarily choose $\theta = \sqrt{3}$. Doing this we expect r to diverge to infinity, which we observe. For long time scales, hyperbolic plots are just straight lines since potential energy converges to much less than the

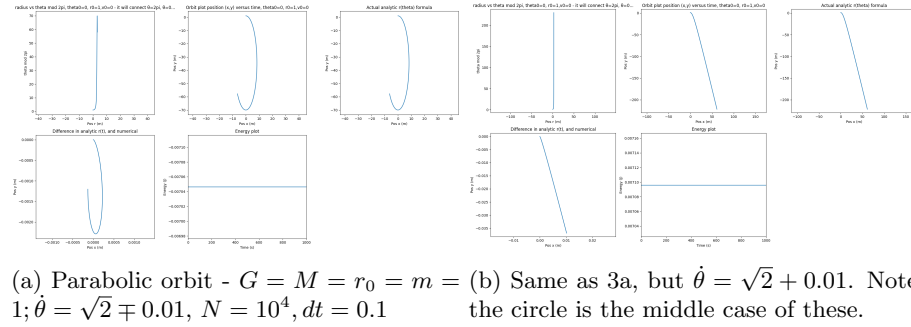
¹To be slightly pedantic, it'd have to be numeric anyways since the Kepler problem is transcendental in most variables

²Numerically we hope to see $dE/dt \approx 0$ for all time, though we expect large E drift for Euler, especially at large time scale (where large means on order compared to the natural orbit time scale generated by the initial conditions). For RK4, we hope to see highly precise results for practically all time scales, though this of course depends on the initial conditions.

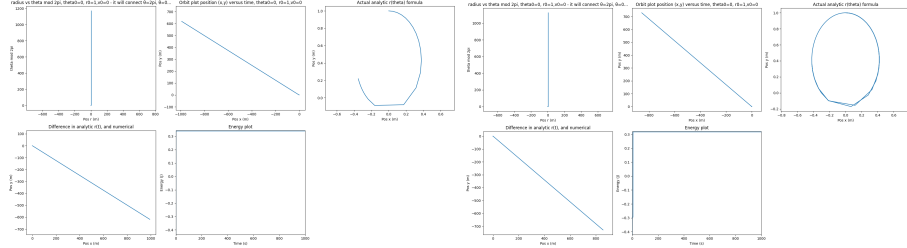
kinetic, at length scales $r \gg r_0$, which we also observe. Figure 2a shows the same analytic $r(t)$ versus numerical $r(t)$.



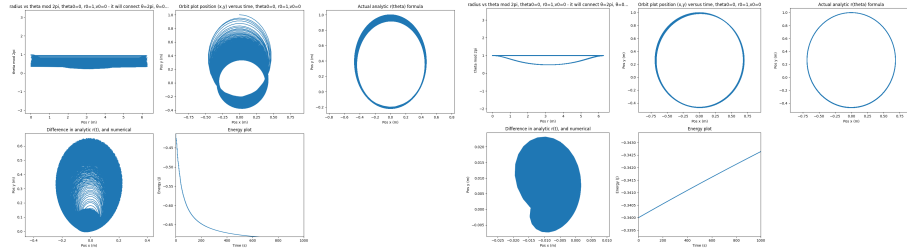
Parabola. For a parabola, we need $E = 0, e = 1$. For this, we require, since $v_0 = 0, \frac{1}{2}\mu r^2 \dot{\theta}_0^2 = \frac{GM\mu}{r}$ which again for $G = M = m = \mu = 1$, some r_0 , we get $\dot{\theta} = \sqrt{\frac{2}{r_0}}$. So do the same as the hyperbola, but take $\dot{\theta}_0 = \sqrt{2}$. Doing this, it's a parabola. We don't plot this, as it's just a limit of hyperbolic orbits, but it is useful to probe the limit $\dot{\theta}_0 = \sqrt{2} \pm \epsilon$ for small epsilon (compared to $\sqrt{2}$). Taking epsilon=0.01, we plot those below in figures ?? to show indeed our numerical results this boundary well. Funny enough too, this boundary is where we observe large error.



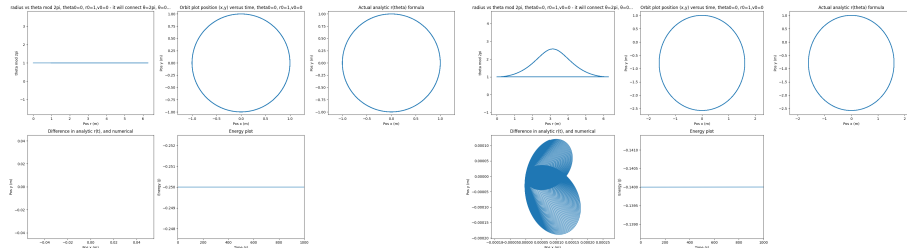
Elliptical. Perhaps the most fun of orbits-elliptical. This is the case of $E < 0, e < 1$. Practically, this is, again for $G = M = m = \mu = 1, r_0 = 1$, equivalent to the requirement that $0 < \dot{\theta}_0 < \sqrt{2}$. We plot the example $\sqrt{2}/2$ to get an eccentricity $e = 0.5$. **insert



(a) Elliptical orbit - $G = M = r_0 = m = 1; \dot{\theta} = 0.5, N = 10^4, dt = 0.1$ (b) Elliptical orbit - $G = M = r_0 = m = 1; \dot{\theta} = 0.55, N = 10^4, dt = 0.1$



(a) Elliptical orbit - $G = M = r_0 = m = 1; \dot{\theta} = 0.6, N = 10^4, dt = 0.1$ (b) Elliptical orbit - $G = M = r_0 = m = 1; \dot{\theta} = 0.8, N = 10^4, dt = 0.1$



(a) Elliptical orbit - $G = M = r_0 = m = 1; \dot{\theta} = 1, N = 10^4, dt = 0.1$ (b) Elliptical orbit - $G = M = r_0 = m = 1; \dot{\theta} = 1.2, N = 10^4, dt = 0.1$

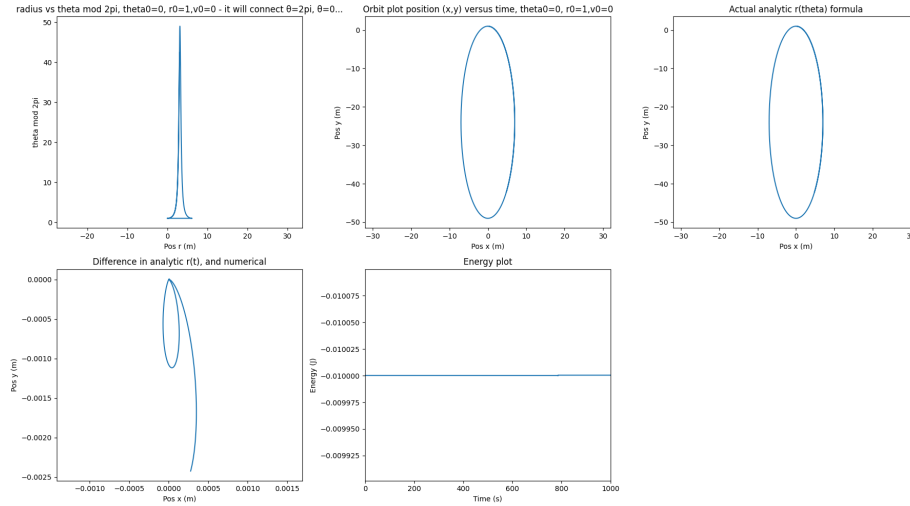
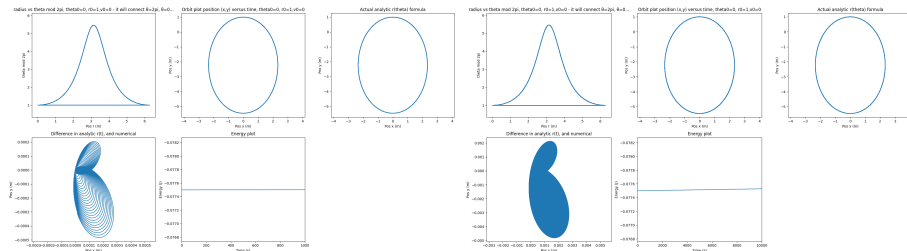


Figure 7: Elliptical orbit - $G = M = r_0 = m = 1; \dot{\theta} = 1.4, N = 10^4, dt = 0.1$. Note that $\dot{\theta} = \sqrt{2} \approx 1.4$ is the non-closed orbit limit, and we see that limit appearing.

On Euler. See the appendix for all comparisons between RK4 and Euler. Each plot compares the Euler ODE solver to the analytic.

4.2 Examples of Large Error

The, perhaps, best example of large error is in taking Ndt to be a large number - e.g. to let the code complete many many orbits, and then observing E over time, for a given orbit type. The plot below is the elliptical orbit, again, but this time for 100x the time - that being $10^6 \cdot 0.1 = 10^5 s$. You can see the accumulated error compared to the above figure 11. There are a multitude of other potential large error cases, and so this is simply a demonstration of one possibility.



(a) Elliptical orbit - $G = M = r_0 = m = 1; \dot{\theta} = 1.3, N = 10^4, dt = 0.1$ (b) Elliptical orbit - $G = M = r_0 = m = 1; \dot{\theta} = 1.3, N = 10^5, dt = 0.1$

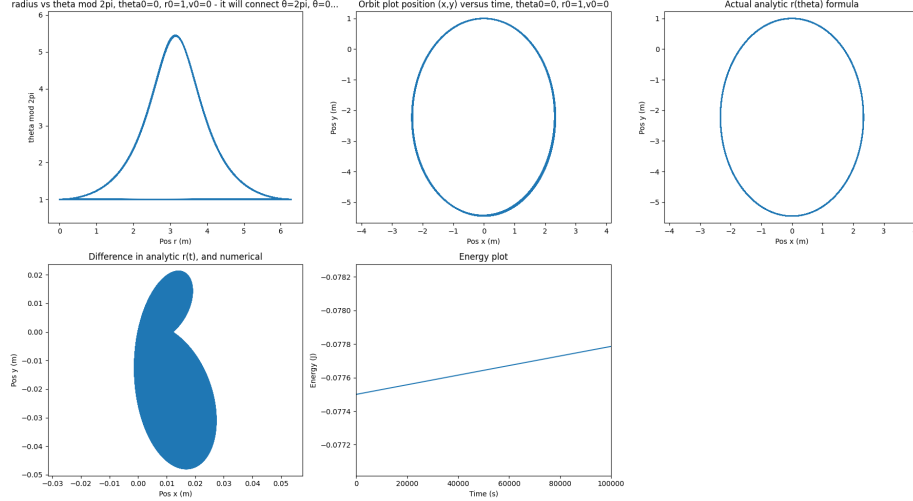
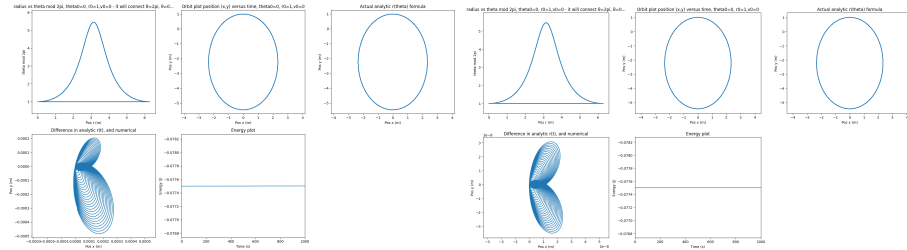


Figure 9: Elliptical orbit - $G = M = r_0 = m = 1; \dot{\theta} = 1.3, N = 10^4, dt = 0.1$. Note the apparent energy drift in the 5th subfigure, now that $t_{\max}$ is large enough.

4.3 Truncation error analytics & behavior parameters

From earlier, our behavior parameter is given as E . We will consider, for simplicity, though any other error could be just as "easily" tracked, the drift of E in the Euler and RK4 schemes at fixed orbit parameters, and variable N, dt . Knowing that the analytic formulas require $dt \rightarrow 0, t = dtN = \int dt \implies N \rightarrow \infty$, we are essentially tracking the error that finite N, dt generate, for the two numerical schemes. Below are 6 plots - three for Euler, three for RK4, at $dt = 0.1, 0.01, 0.001$ for $t = 1000s$, where $N = t/dt$. We take 1 for all variables, except $v_0 = 0$, meaning we're in an elliptical orbit.



(a) Elliptical orbit - $G = M = r_0 = m = 1; \dot{\theta} = 1.3, N = 10^4, dt = 0.1$ (b) Elliptical orbit - $G = M = r_0 = m = 1; \dot{\theta} = 1.3, N = 10^5, dt = 0.01$

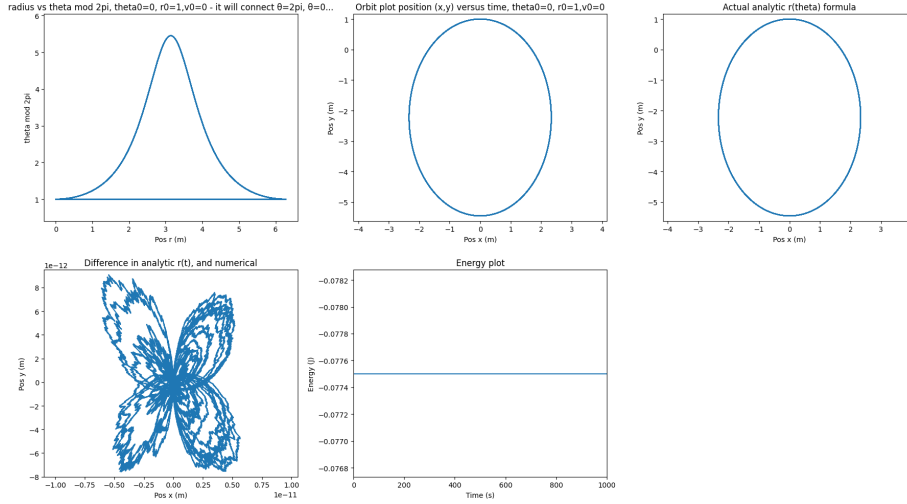


Figure 11: Elliptical orbit - $G = M = r_0 = m = 1; \dot{\theta} = 1.3, N = 10^6, dt = 0.001$. Note the wonderful stochastic like difference in the numerical and analytic, at this very long time scale! (if the energy was to be zoomed in on, the pattern would hold there as well.)

5 Example Solutions - Integration

5.1 Figures

See below Figure 12, the nine (3 with 3 subplots) plots which represent given initial conditions $s, t \in [-3, -1]$, with 10 evenly split points between $-3, -1$ in both, with $N = 100$ evenly split points between 0,1. The first is Riemann integration, second is Trapezoidal, third is Simpson based.

5.2 Examples of Large Error

See Figures 13, 14. Figure 13 is from $z \in [-1, -0.1]$ whereas 14 is over $z \in [-0.1, -0.01]$. Near the boundary $s, t \approx 0$ we note $O(1)$ error, where error is computed as the fractional difference between the numerical, SciPy methods. The integration scheme leads to surprisingly nontrivial differences in error. Note the error in Riemann, Trapezoidal, Simpson. Simpson has the lowest error in every case, with Riemann second lowest error, and Trapezoidal having the third (highest) error consistently.

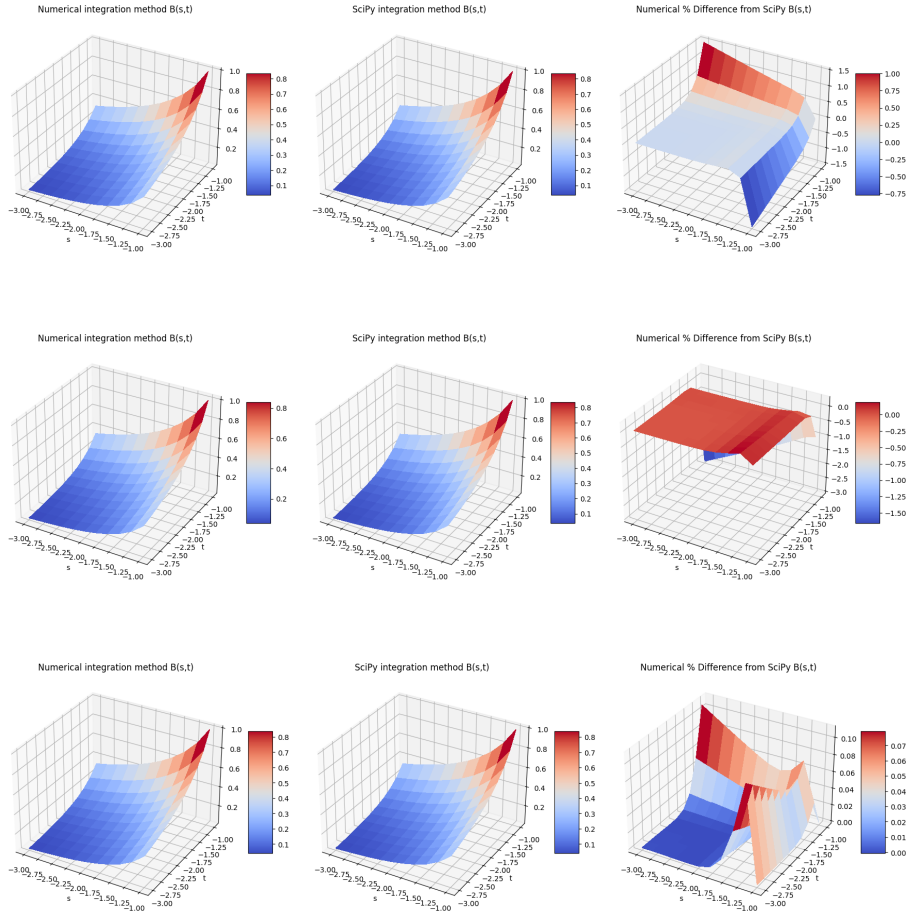


Figure 12: $s, t \in [-3, -1]$, with 10 evenly split points between $-3, -1$ in both, with $N = 100$ evenly split points between $0, 1$. The first is Riemann integration, second is Trapezoidal, third is Simpson based.

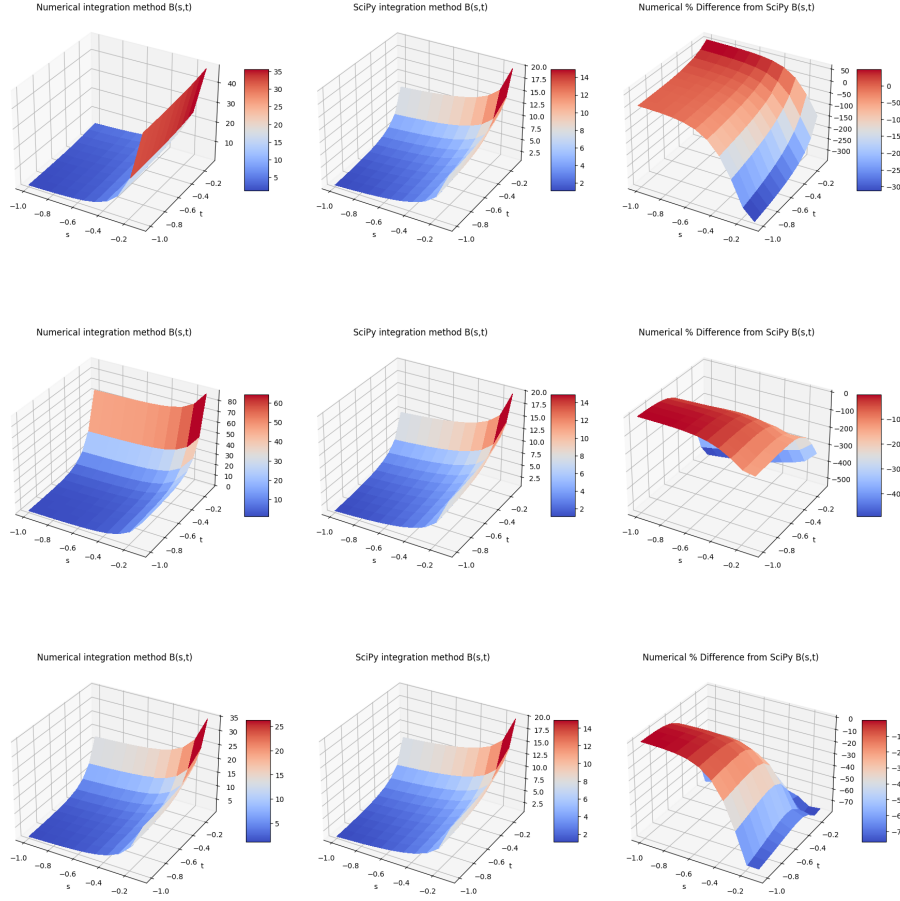


Figure 13: $s, t \in [-1, -0.1]$, with 10 evenly split points between $-3, -1$ in both, with $N = 100$ evenly split points between $0, 1$. The first is Riemann integration, second is Trapezoidal, third is Simpson based. Note the $O(1)$ error in each case as the $s, t \rightarrow 0$ limits are approached.

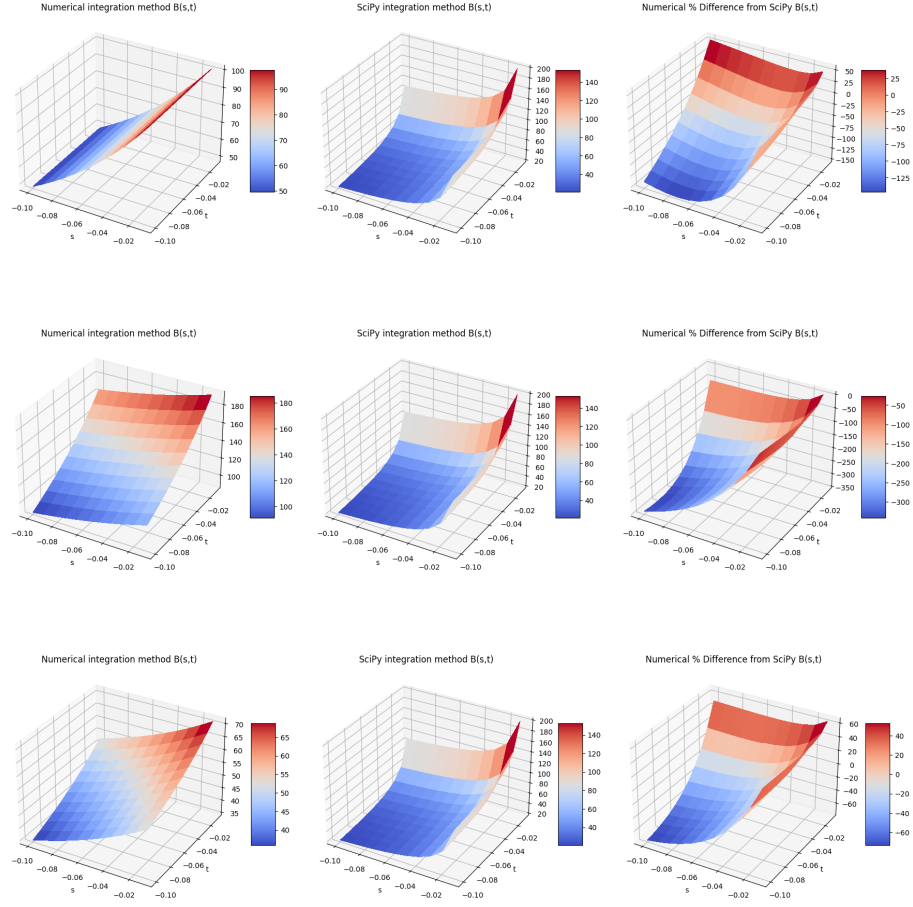


Figure 14: $s, t \in [-0.1, -0.01]$, with 10 evenly split points between $-3, -1$ in both, with $N = 100$ evenly split points between $0, 1$. The first is Riemann integration, second is Trapezoidal, third is Simpson based. Note the $O(1)$ error in each case as the $s, t \rightarrow 0$ limits are approached.

5.3 Truncation error analytics & behavior parameters

See Figures 15, 16, and 17. For the Euler Beta $B(s, t)$, since it is a well-behaved function over our integration domain, we simply then want to be sure that it matches that of SciPy's Euler Beta. To do this, we take the difference, and plot the difference, at grid $s, t \in [-3, -1]$, at 10 s, t points. As for the amount of points, e.g. $1/dz$, the four plots are, in order 10, 100, 500 integration sampling points. Below are various plots that demonstrate the plot of numerical integration, SciPy integration, and their difference. The key here is that we note an approximate $O(10)$ decrease in error, across all three integration schemes, for $N \rightarrow O(10) \cdot N$. In some cases, the error significantly decreases, as you may note from $N = 10 \rightarrow N = 100$ the Simpson error goes from 2.5% to $\approx 0.02\%$.

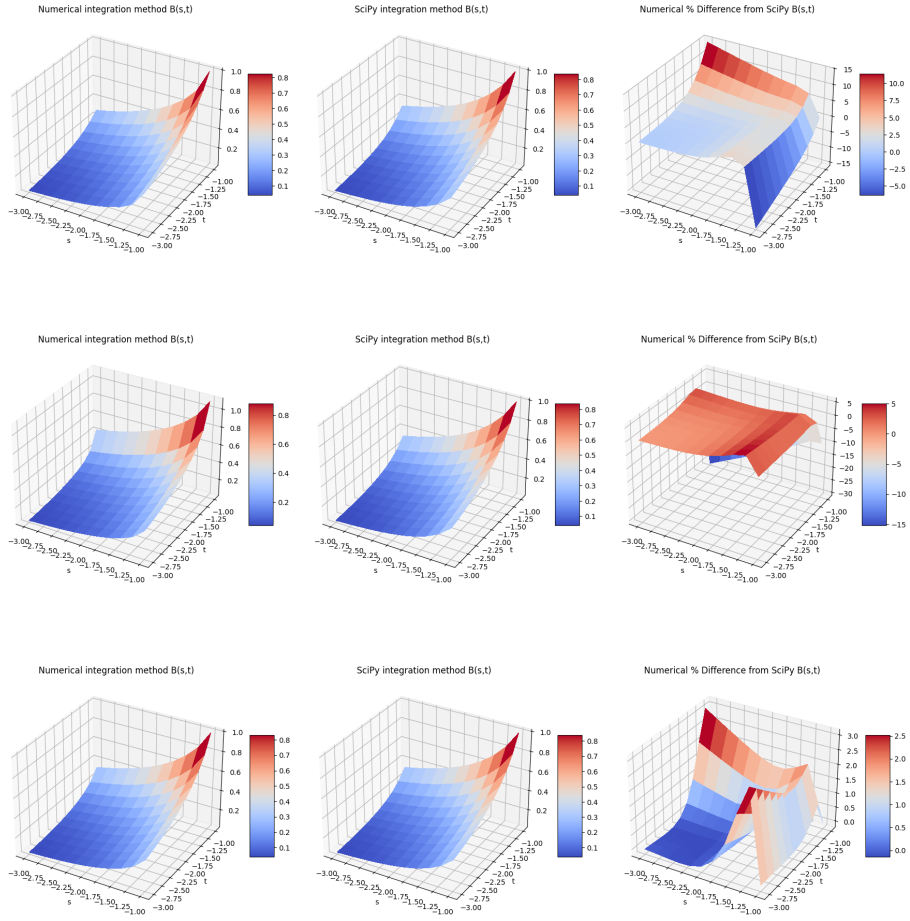


Figure 15: $s, t \in [-3, -1]$, with 10 evenly split points between $-3, -1$ in both, with $N = 10$ evenly split points between $0, 1$. The first is Riemann integration, second is Trapezoidal, third is Simpson based.

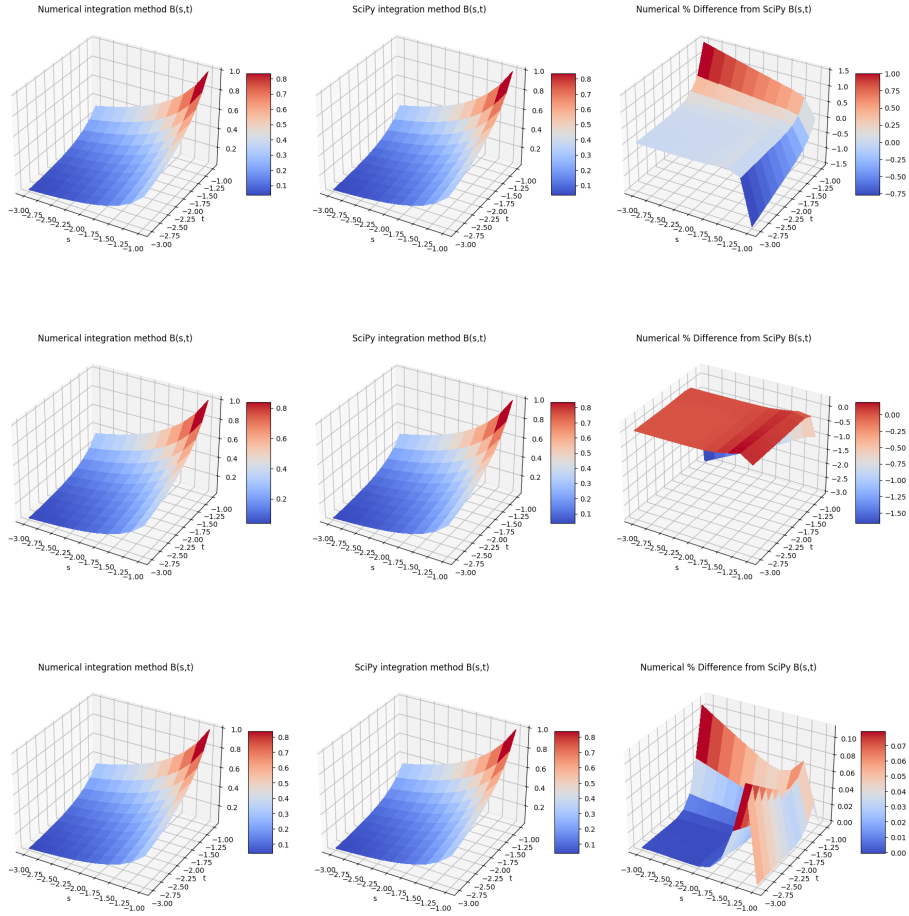


Figure 16: $s, t \in [-3, -1]$, with 10 evenly split points between $-3, -1$ in both, with $N = 100$ evenly split points between $0, 1$. The first is Riemann integration, second is Trapezoidal, third is Simpson based.

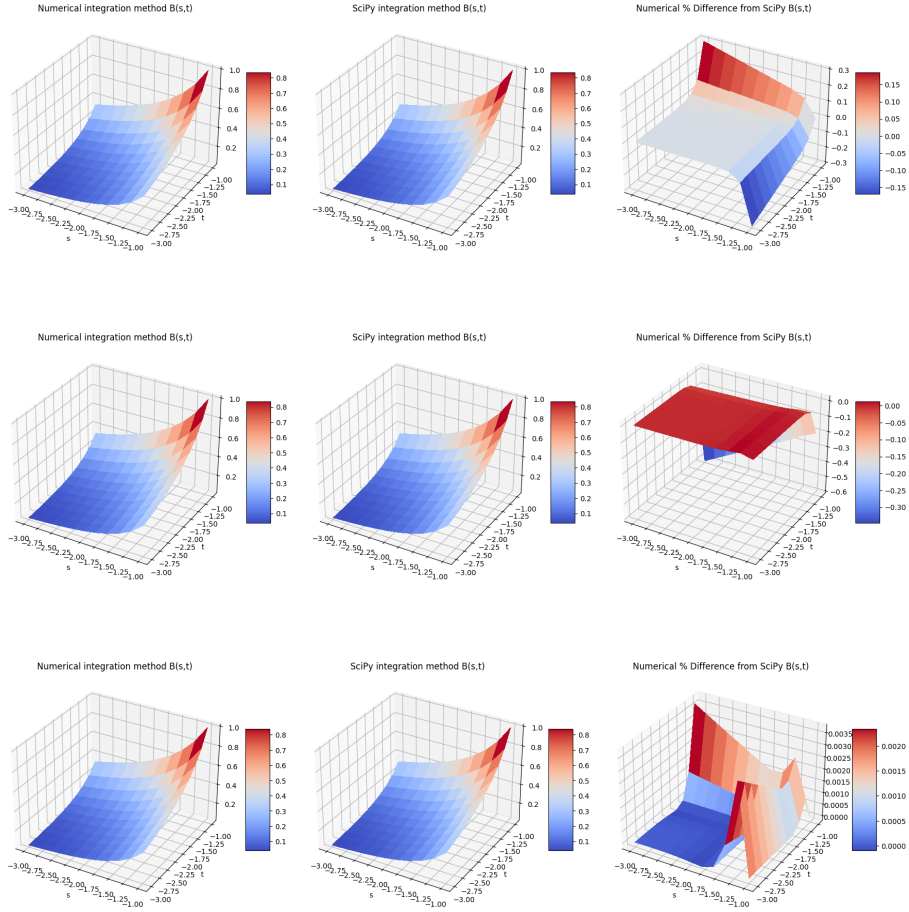


Figure 17: $s, t \in [-3, -1]$, with 10 evenly split points between $-3, -1$ in both, with $N = 500$ evenly split points between $0, 1$. The first is Riemann integration, second is Trapezoidal, third is Simpson based.

6 Using The Scripts Yourself

You need python3 installed, with python3 libraries numpy, matplotlib.pyplot, sympy, scipy.special, matplotlib.ticker. Install both scripts to a location T, in a linux terminal do `cd T`, then `python3 phy607proj1_int.py`, or `python3 phy607proj1_ode.py`. The scripts prompt you, the user, through all steps. All outputs are designed to be detailed enough to be self-explanatory, and figures are labeled similarly. If you have any further questions, or are finding errors, please send all inquiries to gmsuarez@syr.edu.

7 Appendix

7.1 E.1 - Euler ODE

I've actually decided that rather than spam 30 more plots, there are three key plots-the $\dot{\theta} = 1.3$ at varying $t_{\max}$ analysis. Use the figures below and compare to Figure 10, 11. Note Figure 10 is not labeled Figure 10, but rather the figure above Figure 11.

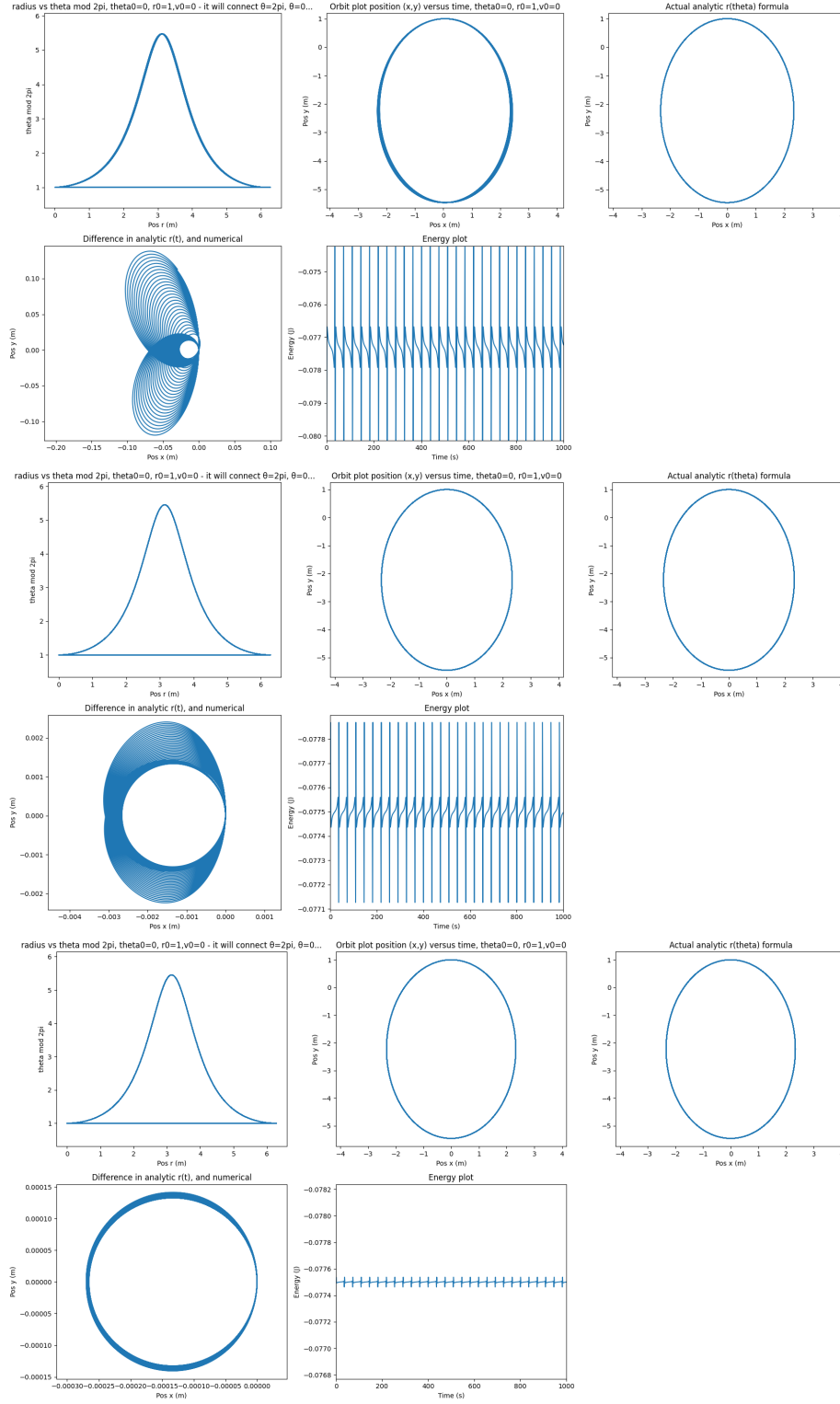


Figure 18: Euler method solver with the exact same parameters of Figure 10, 11 - $G = M = r_0 = m = 1$; $\dot{\theta} = 1.3$. $N = 10^4, 10^5, 10^6$; $dt = 0.1, 0.01, 0.001$. Note the difference in the energy here versus the RK4 energy. With Euler it oscillates-related to the symplectic nature.